□ SOFTWARE TESTING COMPLETE PYQ MASTER GUIDE

IIT Madras | 2023 – 2025 | All Exams Covered
Zero to Hero: Step-by-Step for Beginners

Detail	Info
□ Exams Covered	2023 Dec, 2024 Sep, 2024 Dec, 2025 Apr, 2025 Aug
□ Target Score	100 / 100
□ Total Topics	15 Major Topics with All Sub-topics
□ Important Questions	Frequency-tagged throughout
□ Learning Style	Beginner-Friendly | Step-by-Step | Memory Tips

SUMIT GUPTA MADE THIS PDF WITH USE OF AI

Study Smart. Score 100. □

TABLE OF CONTENTS

#	Topic	Questions	Difficulty
1	Types of Testing (Performance, Security, etc.)	12+	⭑⭑
2	Web Testing & ATG Transitions	10+	⭑⭑⭑
3	Input Space Partitioning (ISP/ECC/BCC)	8+	⭑⭑⭑
4	Control Flow Graphs (CFG)	15+	⭑⭑⭑⭑
5	Data Flow Testing (DU-paths)	8+	⭑⭑⭑⭑
6	Mutation Testing (Basic Operators)	10+	⭑⭑⭑
7	Mutation Testing (OO Operators)	10+	⭑⭑⭑⭑
8	OO Anomalies & Method Invocation	10+	⭑⭑⭑⭑
9	Symbolic Execution & DART	6+	⭑⭑⭑⭑⭑
10	Logic Coverage (RACC/RICC/CACC/GICC)	12+	⭑⭑⭑⭑⭑
11	Component Expressions (Servlet Testing)	6+	⭑⭑⭑⭑
12	Context-Free Grammars & FSM	8+	⭑⭑⭑
13	TDD & Software Quality Metrics	6+	⭑⭑
14	Regular Expressions & Deployment	3+	⭑⭑
15	MEGA CHEAT SHEET & MIND MAP	—	MUST READ

CHAPTER 1: Types of Testing

 APPEARED IN: 2023-Dec, 2024-Sep, 2024-Dec, 2025-Apr, 2025-Aug | HIGH FREQUENCY TOPIC

1.1 Non-Functional Testing Types

These are tests that check HOW WELL the system works, not WHAT it does.

Testing Type	What it Tests	Key Phrase to Remember
Performance Testing	Speed & response time conforming to specs	Speed + Specs
Scalability Testing	Can handle load up to engineering limits	Max Load Limits
Reliability Testing	Keep operating over specified time periods	Time + Uptime
Load Testing	Performance under expected/normal load	Normal Traffic
Stress Testing	Performance beyond normal load (breaking point)	Beyond limits
Spike Testing	Sudden huge increase in users	Sudden surge
Soak/Endurance Testing	Memory leaks over continuous load (long time)	Long time + Memory
Security Testing	Authorized access, virus checks, backdoors	Access Control
Compatibility Testing	Interoperate with 3rd party products	Works with others
Regulatory Testing	Compliance with laws/regulations	Legal compliance
Usability Testing	User-friendliness of the interface	User-friendly

1.2 Functional Testing Levels

Level	Who Tests?	What is Tested?	Memory Key
Unit Testing	Developer	Individual function/class	1 function
Integration Testing	Developer/Team	Modules combined	Modules together
System Testing	QA Team	Full system	Whole product
Acceptance Testing	Client/Internal	Product meets requirements	Client sign-off
Beta Testing	End users OUTSIDE dev team	Near-finished product	Real users
Regression Testing	Dev/QA Team	Old features still work after changes	Nothing broke

1.3 PYQ Questions – Types of Testing

Q1. Which testing of a near-finished software product is done by a set of end users OUTSIDE the development team?

✅**ANSWER: Beta Testing**

→ Key words = "end users" + "outside development team" + "near-finished"

→ Beta Testing = real users (not the dev team) testing a nearly complete product

 MEMORY TIP: BETA = "B" for Beyond the team. Real users test it.

Q2. Which testing verifies that a system can handle load up to its engineering limits?

✅**ANSWER: Scalability Testing**

→ "engineering limits" = how far can it scale?

MEMORY TIP: SCALE = how big can it get? Engineering limits = maximum capacity

Q3. Which test is performed to measure speed and response time conforming to specs?

✅ANSWER: Performance Testing

→ "Speed + response time + conforming to specifications" = Performance Testing

MEMORY TIP: PERFORMANCE = How fast? Think racecar performance.

Q4. Which test monitors system parameters like memory utilization for memory leaks under continuous expected load?

✅ANSWER: Soak Testing (Endurance Testing)

→ "continuous expected load" + "memory leaks" + "long time" = Soak Testing

MEMORY TIP: SOAK = soaking = long time in water. Memory leaks over long run.

Q5. Which test suddenly increases users to a very large amount to measure performance?

✅ANSWER: Spike Testing

→ "SUDDENLY increasing" users = Spike Testing

MEMORY TIP: SPIKE = sharp sudden spike on a graph. Flash sale = spike!

Q6. Which of the following is NOT a security testing technique?

✅ANSWER: To measure ability of system to keep operating over specified periods of time (this is Reliability Testing!)

→ Security testing includes: authorized access, virus checkers, identifying backdoors

→ "Operating over time" = Reliability Testing, NOT security testing

MEMORY TIP: Security = PROTECTION. Reliability = UPTIME. Do not confuse them!

Q7. Which are considered as security testing? (Multiple Select)

✅ANSWER: 1) Verify only authorized accesses permitted 2) Ensure virus checkers prevent/curtail viruses

→ Security = Access Control + Virus Protection

→ NOT security: interoperability (compatibility), keeping operating over time (reliability)

Q8. Which testing involves testing the LIMIT of the system - magnitude of demand while meeting latency/throughput?

✅ANSWER: Scalability Testing

→ "Limit of system" + "magnitude of demand" + "latency/throughput" = Scalability Testing

CHAPTER 2: Web Testing & Application Transition Graph (ATG)

APPEARED IN: ALL 5 EXAMS | VERY HIGH FREQUENCY – MASTER THIS!

2.1 What is an ATG?

An Application Transition Graph (ATG) models HOW a web application moves from one page/state to another. Each NODE = a web component (page). Each EDGE = a transition (how you got from A to B).

2.2 Types of ATG Transitions

Transition Type	How it happens	Real-world Example	Key Identifier
Simple Link Transition	User clicks a hyperlink (href)	Click "About Us" link	hyperlink / href / URL edit
Form Link Transition	User fills form and submits	Login form submit	Form submission
Redirect Transition	Server/system automatically redirects	After payment, return to store	Automatic/programmatic redirect
Operational Transition	Back/Forward browser button OR JavaScript	Back/Forward button	Back/Forward buttons

2.3 PYQ Questions – ATG Transitions

Q1. User makes online payment through payment gateway. After success, user is SENT BACK to e-commerce with order confirmation. Which transition type?

✅ **ANSWER: Redirect Transition**

→ Was this a link the user clicked? NO

→ Was this a form submission? NO

→ The system AUTOMATICALLY sent the user back = Redirect Transition

MEMORY TIP: REDIRECT = payment gateway REDIRECTS you back. No user action needed.

Q2. ATG has a transition triggered when user MANUALLY EDITS the URL. Which type?

✅ **ANSWER: Simple Link Transition**

→ User manually types/edits URL = same as clicking a link

MEMORY TIP: SIMPLE LINK = URL bar edit = same as clicking. Direct navigation.

Q3. User fills a web form and clicks Submit. App transitions to confirmation page. Which ATG transition?

✅ **ANSWER: Form Link Transition**

→ User filled a FORM and clicked SUBMIT = Form Link Transition

MEMORY TIP: FORM LINK = Form + Submit button = Form Link Transition

Q4. User CLICKS ON A HYPERLINK and browser navigates to different page. Which ATG transition?

✅ **ANSWER: Simple Link Transition**

→ Hyperlink click = Simple Link Transition. This is the most basic navigation.

Q5. Transitions between pages using BACK and FORWARD browser buttons. Which type?

✅**ANSWER: Operational Transition**

→ Back/Forward buttons are browser operations = Operational Transition

MEMORY TIP: OPERATIONAL = browser OPERATIONs = Back/Forward/Refresh buttons

2.4 Web Application Layers

Layer	Purpose	Memory Key
1. Presentation Layer	User Interface (HTML/CSS)	What you SEE
2. Data Content Layer	Collection of web pages linked by content/syntax	The website structure
3. Data Representation Layer	Computation and data access logic	Processing/Logic
4. Data Storage Layer	Permanent data storage (database)	What is STORED permanently

2.5 Client-Side Bypass Testing Types

Type	What it tests	Key Words	Example
Value Level Bypass Testing	Invalid input values / inter-value constraints	Invalid inputs, credit card + expiry date	Testing if negative age is rejected
Parameter Level Bypass Testing	Relationships between different parameters	Relationships among parameters	Check-in date vs Check-out date
Control Flow Level Bypass Testing	Breaking normal execution sequence	Normal execution sequence	Skip login step
User-Session Data Based Testing	Data captured during user sessions	Session data, user sessions	Replay a recorded session

Q6. Which client-side testing checks issues related to RELATIONSHIPS AMONG DIFFERENT PARAMETERS?

✅**ANSWER: Parameter Level Bypass Testing**

MEMORY TIP: PARAMETER = relationship between PARAMS. Like: checkout date must be AFTER checkin date.

Q7. An online booking system – tester alters check-out date to be EARLIER than check-in date. What type of testing?

✅**ANSWER: Parameter Level Bypass Testing**

→ Checkin date and checkout date are TWO PARAMETERS whose RELATIONSHIP is being tested

Q8. Which client-side testing VERIFIES IF WEB APP ADEQUATELY EVALUATES INVALID INPUTS?

✅**ANSWER: Value Level Bypass Testing**

→ "Invalid inputs" + "evaluate" = Value Level Bypass Testing

MEMORY TIP: VALUE = the actual input value. Invalid = wrong value. Value testing!

Q9. Which client-side testing tries to verify web apps by executing test cases that BREAK THE NORMAL EXECUTION SEQUENCE?

✅**ANSWER: Control Flow Level Bypass Testing**

MEMORY TIP: CONTROL FLOW = flow of execution. Breaking it = Control Flow Bypass.

Q10. Which needs to verify INTER-VALUE CONSTRAINTS like credit card number and the expiry date?

✅**ANSWER: Value Level Bypass Testing**

→ "Inter-value constraints" = relationship between VALUES = Value Level

CHAPTER 3: Input Space Partitioning (ISP/ECC/BCC)

APPEARED IN: 2023-Dec, 2024-Sep, 2024-Dec, 2025-Aug | CALCULATION BASED

3.1 Basic Concepts

Input Space Partitioning divides inputs into BLOCKS (groups). Each block should behave the same way.

CHARACTERISTIC = a property/feature of the input (like: is it negative? is it zero?)

BLOCK = a group of values for that characteristic

3.2 Coverage Criteria

Criterion	What it requires	Formula	Example
Each Choice Coverage (ECC)	Each block appears in at least ONE test case	Max block count across all characteristics	If [A,B,C],[X,Y] need max(3,2)=3 tests
Base Choice Coverage (BCC)	One base test case + vary one at a time	1 + (total_blocks - num_chars)	Calculated below
All Combinations (ACoC)	Every combination of all blocks	Product of all block counts	HUGE number of tests

3.3 ECC Formula – Step by Step

ECC Tests = Maximum number of blocks across all characteristics

Example: Characteristics = [A,B,C,D], [1,2,3], [x,y]

Block counts = 4, 3, 2 --> ECC = max(4, 3, 2) = 4 tests

Q1. Three partitions [A,B,C,D], [1,2,3], [x,y]. Minimum number of tests using Each Choice Coverage (ECC)?

✅ **ANSWER: 4 tests**

→ Count blocks per characteristic: [A,B,C,D]=4, [1,2,3]=3, [x,y]=2

→ ECC = Maximum of all block counts = max(4,3,2) = 4

MEMORY TIP: ECC = just the MAXIMUM block count.

Q2. Four partitions [X1,X2], [Y1,Y2,Y3,Y4], [W1,W2,W3], [Z1,Z2]. ECC minimum tests?

✅ **ANSWER: 4 tests (max of 2,4,3,2 = 4)**

→ Block counts: X=2, Y=4, W=3, Z=2 --> ECC = max(2,4,3,2) = 4

3.4 BCC Formula – Step by Step

BCC Total Tests = 1 (base test) + sum of (blocks_i - 1) for each characteristic i

Alternative formula: BCC = 1 + (total blocks) - (number of characteristics)

Q3. ISP with 4 blocks [A1,A2,A3,A4], [B1,B2], [C1,C2,C3], [D1,D2]. Base choice = (A1,B2,C3,D2). Total BCC tests?

✅ **ANSWER: 8 tests**

→ Count blocks per characteristic: A=4, B=2, C=3, D=2

→ BCC = 1 (base) + (4-1) + (2-1) + (3-1) + (2-1)

→ = 1 + 3 + 1 + 2 + 1 = 8

MEMORY TIP: BCC = 1 + (A-1) + (B-1) + (C-1) + (D-1) = 1 + extra blocks for each characteristic

Q4. Annuities: investedAmount(3 blocks), yearsOfInvestment(2 blocks), gender(3 blocks), isSeniorCitizen(2 blocks). ECC tests?

✅**ANSWER: 3 tests (max of 3,2,3,2 = 3)**

3.5 BVA – Boundary Value Analysis

BVA Type	Test Values	Why
Normal BVA	{58, 68, 78}	Min, mid, max
Robust BVA	{57, 58, 68, 78, 79}	One below min, min, mid, max, one above max
Worst Case BVA	{57, 58, 59, 68, 77, 78, 79}	All boundary points

Q5. Partition 58 <= candidate_weight <= 78. Valid set for BVA?

✅**ANSWER: Robust BVA: {57, 58, 68, 77, 78}**

→ Robust BVA = min-1, min, nominal, max, max+1

CHAPTER 4: Control Flow Graphs (CFG) – Coverage Criteria

 APPEARED IN: ALL EXAMS | MOST IMPORTANT CHAPTER – 15+ Questions!

4.1 What is a CFG?

A Control Flow Graph represents ALL POSSIBLE PATHS through a program.

NODE = statement or decision point. EDGE = flow from one node to another.

4.2 Coverage Criteria (Weakest to Strongest)

Coverage	What must be covered	Min test paths
Node Coverage (NC)	Every NODE visited at least once	Usually 1-2 paths
Edge Coverage (EC)	Every EDGE traversed at least once	Usually 2+ paths
Edge-Pair Coverage (EPC)	Every consecutive pair of edges traversed	More paths needed
Prime Path Coverage (PPC)	Every PRIME PATH covered (with side trips/detours)	Most paths needed

4.3 Cyclomatic Complexity

V(G) = E - N + 2 (E = edges, N = nodes)

OR: V(G) = Number of regions in the graph = Number of binary decisions + 1

Q1. CFG G: V={1,2,3,4,5,6,7,8}, E={(1,2),(1,3),(2,4),(3,4),(4,5),(4,7),(5,6),(5,7),(6,8),(7,8)}. Cyclomatic complexity?

✅ **ANSWER: 4**

→ Count edges E = 10

→ Count nodes N = 8

→ V(G) = E - N + 2 = 10 - 8 + 2 = 4

 MEMORY TIP: FORMULA: V(G) = E - N + 2. Count every arrow (edge) and every circle (node).

Q2. CFG: V={1,2,3,4,5,6,7}, E={(1,2),(2,3),(2,4),(3,4),(4,5),(4,7),(5,6),(5,7),(6,7)}. Cyclomatic complexity?

✅ **ANSWER: 4 (E=9, N=7, V(G) = 9 - 7 + 2 = 4)**

Q3. CFG: V={1,2,3,4,5}, E={(1,2),(2,3),(2,5),(3,4),(3,5),(4,3)}. Cyclomatic complexity?

✅ **ANSWER: 3 (E=6, N=5, V(G) = 6 - 5 + 2 = 3)**

Q4. factorial() function with if(n>=0), if(n<2), for loop. Cyclomatic complexity?

✅ **ANSWER: 4**

→ Decisions: if(n>=0), if(n<2), for(i<=n) = 3 decisions + 1 = 4

4.4 Edge-Pair Coverage (EPC)

Edge-Pair = a sequence of 2 consecutive edges (3 nodes): [A->B->C]

Count ALL possible 2-edge paths that exist in the graph.

Q5. CFG same as Q1. Edge-pair coverage test requirements?

✅**ANSWER: 11**

→ For each node, find all (incoming edge, outgoing edge) pairs

→ Systematically count all [x->y->z] triples. Total = 11

MEMORY TIP: EPC = For each node with multiple in AND out edges, multiply them.

Q6. CFG: V={1..7}, E={(1,2),(1,3),(2,4),(3,4),(3,5),(4,6),(4,7),(5,3),(6,7)}. Edge-pair coverage?

✅**ANSWER: 9**

→ Node 4: in-edges from 2,3; out-edges to 6,7 --> 2x2=4 pairs

→ Node 3: in-edges from 1,5; out-edges to 4,5 --> 2x2=4 pairs

→ Plus starting paths from node 1 --> total = 9

4.5 Prime Paths

A PRIME PATH is a path that: (1) has NO repeated nodes, (2) cannot be extended (maximal)

Touring Method	Meaning	Example for P=[1,2,4,5]
Directly (no side trips)	Test path IS exactly the prime path	[1,2,4,5] identical
With a Side Trip	Includes extra loops but returns to same node	[1,2,3,2,4,5] 2->3->2 loop
With a Detour	Takes different route through a node	[1,2,3,4,5] goes through 3 instead

Q7. Prime path P=[1,2,4,5]. Among [1,2,4,2,4,5], [1,2,4,2,3,4], [1,2,3,2,4,5], [1,2,3,4,5]. Which tours P with a SIDETRIP but NOT a detour?

✅**ANSWER: [1,2,3,2,4,5]**

→ P = [1,2,4,5]. Must visit nodes 1,2,4,5 in order

→ [1,2,3,2,4,5] visits 1,2,4,5 correctly but loops through 3. Node 2 is visited TWICE.

→ The extra 2->3->2 is a side trip (returns to same node 2). This tours P with a side trip

→ [1,2,3,4,5] is a DETOUR - never visits the 2->4 edge directly

MEMORY TIP: SIDE TRIP = detour that RETURNS to where it started from. Like a side road that brings you back.

Q8. Min test paths for Node and Edge Coverage for CFG V={1,2,3,4,5}, E={(1,2),(2,3),(2,5),(3,4),(3,5),(4,3)}?

✅**ANSWER: Node Coverage = 1 or 2; Edge Coverage = 2**

→ Node coverage: find a path that visits ALL nodes

→ Edge coverage: find paths that traverse ALL edges (may need 2 paths if there is a loop)

MEMORY TIP: NODE COVERAGE <= EDGE COVERAGE <= PRIME PATH COVERAGE. Always true!

CHAPTER 5: Data Flow Testing (DU-Paths)

 APPEARED IN: ALL 5 EXAMS | HIGH FREQUENCY

5.1 Key Definitions

Term	Meaning	Example
DEF (definition)	Variable gets a VALUE assigned	x = 5 --> x is DEFINED at this node
USE (use)	Variable VALUE is READ	if(x > 0) --> x is USED here
DU-Pair	A def of variable x + a use of variable x	(def at node 1, use at node 3)
DU-Path	A path from def to use with NO re-definition in between	Path [1,2,3] if x defined at 1, used at 3, not redefined in 2
Def-Clear Path	A path where variable is NOT redefined	x defined at 1, path 1->2->3, x used at 3 with no new def

5.2 Coverage Criteria

Criterion	What must be satisfied	Strength
All-Defs Coverage	At least ONE use reachable from EACH def of every variable	Weakest
All-Uses Coverage	For each def, EVERY USE reachable is covered by at least one test path	Medium
All-DU-Paths Coverage	ALL distinct DU-paths for every def-use pair are covered	Strongest

All-Defs ⊆ All-Uses ⊆ All-DU-Paths (each stronger than previous)

Q1. For a variable with def at node 1, uses at nodes 3 and 5. Test paths {[1,3,5],[1,2,3,5],[1,3,4,5]}. Coverage for variable?

✅**ANSWER: All-Uses Coverage (not All-DU-Paths if some DU-paths are not covered)**

→ Find DEFs: variable assigned at node 1

→ Find USEs: variable read at nodes 3 and 5

→ All-Uses = every def-use PAIR covered by some path

 MEMORY TIP: TRICK: All-Uses = every def-use PAIR covered. All-DU-Paths = every distinct PATH between each pair.

Q2. Which of the following is a DEF-CLEAR PATH for a variable?

✅**ANSWER: The path from variable definition node to use node where the variable is NOT redefined along the way**

→ Identify where variable is defined (def node)

→ Identify where variable is used (use node)

→ A def-clear path from def->use should not have any other assignment to that variable

CHAPTER 6: Mutation Testing – Basic Operators

APPEARED IN: ALL EXAMS | MEDIUM-HIGH FREQUENCY

6.1 What is Mutation Testing?

Mutation testing checks the QUALITY of your test suite by:

→ 1. Creating slightly modified (mutated) versions called MUTANTS
→ 2. Running your test suite on each mutant
→ 3. If test suite detects the change = mutant is KILLED (good!)
→ 4. If test suite does not detect = mutant SURVIVES (bad - weak test!)

6.2 Types of Mutants

Type	Description	Can it be killed?
Live Mutant	Mutant that has NOT been killed by any test	Yes - but has not been yet
Dead Mutant	Mutant that IS killed by a test	Already killed
Stillborn Mutant	Syntactically invalid - will not even compile	No (invalid code)
Trivial Mutant	Almost every test kills it (too easy)	Yes - easily
Equivalent Mutant	Produces SAME output as original (undetectable)	NO - impossible to kill

Q1. Which type of mutant is VALID and can be KILLED by a test case?

✅**ANSWER: Dead Mutant (or Live Mutant before being killed) - also Trivial Mutant**

→ Stillborn = invalid code, cannot run
→ Equivalent = same behavior, cannot be killed

6.3 Basic Mutation Operators

Operator	What it does	Example
Arithmetic Operator Replacement (AOR)	Replace +,-,*,/,% with each other	x % 2 --> x / 2
Relational Operator Replacement (ROR)	Replace >,<,>=,<=,==,!= with each other	x > 0 --> x >= 0
Conditional Operator Replacement (COR)	Replace && , || with each other	a && b --> a || b
Scalar Variable Replacement (SVR)	Replace one variable with another	z = a + b --> z = c + b
Absolute Value Insertion (AVI)	Insert abs() around expression	x + y --> abs(x) + y
Statement Deletion (SDL)	Remove a whole statement	delete entire line

Q2. Ground string: int x = x % 2; --> Mutant-1: int x = x / 2. Which operator?

✅**ANSWER: Arithmetic Operator Replacement (AOR)**

→ % (modulo) was replaced by / (division)
→ Both % and / are arithmetic operators --> AOR

*MEMORY TIP: ARITHMETIC = math operators: + - * / %. If one is swapped for another = AOR*

Q3. Ground: int z = (a+b)*(a-b); --> Mutant: int z = (a+a)*(a-b). Which operator?

✅**ANSWER: Scalar Variable Replacement (SVR)**

→ The variable "b" in "(a+b)" was replaced by "a"

MEMORY TIP: SVR = Variable replaced by Variable. Same TYPE, different name.

Q4. Ground: return (-b + sqrt(b*b - 4*a*c)) / (2*a). Mutant: return (-b + sqrt(b*b - 4*a*b)) / (2*a). Which?

✅**ANSWER: Scalar Variable Replacement - "c" replaced by "b"**

Q5. While(left <= right) --> Mutant: while(left < right). Which mutation operator?

✅**ANSWER: Relational Operator Replacement (ROR)**

→ <= was replaced by < = relational operators changed = ROR

6.4 Killing a Mutant: RIP Model

To KILL a mutant: REACH --> INFECT --> PROPAGATE

→ REACH: Test execution must REACH the mutated statement

→ INFECT: Execution of mutant must cause a different INTERNAL STATE

→ PROPAGATE: The difference in state must cause different OUTPUT

CHAPTER 7: OO Mutation Operators

 APPEARED IN: ALL EXAMS | HIGH FREQUENCY – 10+ Questions!

Operator	Abbr.	What it does	Example
Parent Constructor Deletion	PCD	Remove super() call from child constructor	Delete "super(value);"
Super Keyword Deletion	SKD	Remove "super." prefix from method call	super.draw() --> draw()
Overriding Method Deletion	OMD	Delete an overriding method entirely	Remove draw() from Circle class
Hiding Variable Insertion	HVI	Add a new variable that hides parent variable	Add "int x;" in subclass
Hiding Variable Deletion	HVD	Remove the hiding variable declaration	Remove "int x;" from subclass
Overloading Method Change	OMC	Change overloaded method to use different type	Method signature type change
Argument Order Change	AOC	Swap order of arguments	sum(10,20.5) --> sum(20.5,10)

Q1. class Precision extends Metric { Precision(double val) { super(value); } }. Mutant created by DELETING super(value). Which operator?

✅**ANSWER: Parent Constructor Deletion (PCD)**

→ super(value) = calling PARENT CONSTRUCTOR

→ Deleting the super() call = Parent Constructor DELETION = PCD

 MEMORY TIP: PCD = deleting super() in CONSTRUCTOR. The "super" is calling parent constructor.

Q2. class Circle extends Shape { draw() { super.draw(); println("Drawing circle"); } }. Mutant deletes super.draw(). Which operator?

✅**ANSWER: Super Keyword Deletion (SKD)**

→ super.draw() calls the parent method using "super"

→ Deleting this super.METHODCALL = Super Keyword Deletion = SKD

 MEMORY TIP: SKD = deleting super.method() - not a constructor, but a method call with super.

Q3. super.x = n changes to x = n (removing "super."). Which operator?

✅**ANSWER: Super Keyword Deletion (SKD)**

→ super.x = n --> x = n means the "super." keyword was removed

→ Now it assigns to child own x, not parent x

Q4. Math class: sum(double a, int b). m.sum(10, 20.5) --> Mutant: m.sum(20.5, 10). Which operator?

✅**ANSWER: Argument Order Change (AOC)**

→ Arguments were SWAPPED: (10, 20.5) --> (20.5, 10)

 MEMORY TIP: AOC = Arguments ORDER Changed = parameters swapped in position

Q5. class Calculator: calculate(double,double) returns num1+num2; calculate(int,int) returns num1*num2. Mutant: calculate(double,double) returns num1*num2. Which operator?

✅**ANSWER: Overloading Method Change (OMC)**

→ The OVERLOADED method body was changed (+ --> *)

 MEMORY TIP: OMC = same method name, different signatures (overloaded), body changed

CHAPTER 8: OO Anomalies & Method Invocation

 APPEARED IN: ALL EXAMS | HIGH FREQUENCY – Memorize the 4 types!

8.1 Four Types of OO Anomalies

Anomaly	Abbr.	Description	Output Issue
State Visibility Anomaly	SVA	Subclass HIDES parent variable by declaring same name. print() uses parent var which is still 0.	prints parent value (e.g. 0 instead of 35)
State Definition Anomaly	SDA	An overriding method does NOT call super, so parent variable is NEVER SET.	parent variable stays at default value
State Definition Inconsistency Anomaly	SDIA	Variable defined in one class but used incorrectly in hierarchy	Inconsistent values
Inconsistent Type Use	ITU	A variable of wrong type is used (type mismatch)	Type-related wrong results

 KEY DISTINCTION: SVA = HIDING by re-declaring variable. SDA = NOT CALLING super method.

8.2 Solving Anomaly Questions – Step by Step

→ STEP 1: Read the SYMPTOM - what is the wrong output?
→ STEP 2: Identify if a variable is being HIDDEN (same name in parent and child) --> SVA
→ STEP 3: Check if an overriding method does NOT call super.method() --> SDA
→ STEP 4: If the output shows 0 or default value when it should be set --> usually SDA
→ STEP 5: If output shows parent value instead of child value --> usually SVA

Q1. Laptop::setStockQuantity does NOT call Product::setStockQuantity, so stockQuantity stays 0. Anomaly?

✅**ANSWER: State Definition Anomaly (SDA)**

→ The overriding method does NOT call super
→ Therefore, Item stockQuantity is never set (it is defined in Item but never updated)

 MEMORY TIP: SDA = overriding method forgot to call super.method(). Variable definition incomplete.

Q2. Class Z extends Y, declares protected int b (same as class X). setB(20) sets Z b, but print() in X prints X b (still 0). Anomaly?

✅**ANSWER: State Visibility Anomaly (SVA)**

→ Z declares "protected int b" = SAME NAME as X "protected int b"
→ Z b HIDES X b. setB(20) sets Z b, but X::print() reads X b (= 0)

 MEMORY TIP: SVA = child HIDES parent variable by re-declaring it. Like shadows!

Q3. Captain extends Player extends Person. Captain declares "protected String name" (same as Person). setName sets Captain name but print() reads Person name. Anomaly?

✅**ANSWER: State Visibility Anomaly (SVA)**

→ Captain re-declares "name" variable --> hides Person::name
→ setName sets Captain name, but Player::print() reads Person name --> SVA

Q4. C::setY sets local "int z = val*3" but NEVER sets A y. Output: x=18, y=0. Anomaly?

✅**ANSWER: State Definition Anomaly (SDA)**

→ C::setY overrides but creates a LOCAL variable z and never calls super.setY
→ A y is NEVER defined = State Definition Anomaly

8.3 Method Invocation in Inheritance

In Java, method resolution follows DYNAMIC DISPATCH (runtime polymorphism):

→ 1. Look at the ACTUAL OBJECT TYPE (not the declared type)
→ 2. Start from the actual class, go UP the hierarchy until method found

Q5. Z extends Y extends X. Z obj = new Z(). obj.c(), obj.d(), obj.e(), obj.f(). Which methods invoked?

✅ **ANSWER: {Z::c(), Y::d(), X::e(), Z::f()}**

→ obj is Z type. Look at Z first.
→ c() - Z has c() --> Z::c()
→ d() - Z has no d(). Y has d() --> Y::d()
→ e() - Z has no e(). Y has no e(). X has e() --> X::e()
→ f() - Z has f() --> Z::f()

MEMORY TIP: DYNAMIC DISPATCH: Start from the actual object type (bottom). Walk UP until method found.

Q6. Z extends Y extends X. flag=true --> obj=new Z(). obj.setA(10), setB(20), getA(), getB(). Which set invoked?

✅ **ANSWER: {Z::setA(), Y::setB(), Z::getA(), X::getB()}**

→ flag=true --> obj is of type Z
→ setA(): Z has setA --> Z::setA()
→ setB(): Z has no setB. Y has setB --> Y::setB()
→ getA(): Z has getA --> Z::getA()
→ getB(): Z has no getB. Y has no getB. X has getB --> X::getB()

CHAPTER 9: Symbolic Execution & DART

 APPEARED IN: ALL 5 EXAMS | HARD BUT CONSISTENT

9.1 What is Symbolic Execution?

Instead of running code with ACTUAL values, use SYMBOLIC values (like algebra).
x0, y0, z0 represent SYMBOLIC inputs. Build up PATH CONSTRAINTS at each decision.

9.2 How to Build Path Constraints

→ 1. Replace input variables with symbols: x --> x0, y --> y0, z --> z0
→ 2. Compute symbolic values: if m = (x+2)*(x-2), then m_sym = x0^2 - 4
→ 3. At each IF condition, record the condition as a constraint
→ 4. For a specific PATH, AND all constraints along that path
→ 5. For the FALSE branch, NEGATE the condition

Q1. void verify(int x) { m=(x+2)*(x-2); if(!isPositive(x)) { if(m<=20) { abort(); }}}. Path constraints to reach abort()?

✅ANSWER: (x0 <= 0) AND ((x0 * x0) - 4 <= 20)

→ isPositive(x) = (x > 0). !isPositive(x) means (x <= 0) --> constraint: x0 <= 0
→ m = (x+2)*(x-2) = x^2 - 4 symbolically --> m_sym = x0*x0 - 4
→ if(m <= 20) is TRUE to reach abort --> constraint: (x0*x0 - 4) <= 20
→ AND both constraints: (x0 <= 0) AND (x0*x0 - 4 <= 20)

 MEMORY TIP: SYMBOLIC: just replace x with x0, compute the expression algebraically!

Q2. check(int x): k=(x+1)*(x-1); if(!isEven(x)) { if(k>10) { abort(); }}. Path to abort()?

✅ANSWER: (x0 % 2 != 0) AND ((x0 * x0) - 1 > 10)

→ isEven(x) = (x%2==0). !isEven --> x%2 != 0
→ k = (x+1)*(x-1) = x^2-1 = x0*x0 - 1
→ k > 10 --> (x0*x0 - 1) > 10

Q3. evaluate(x,y,z): t1=squareIt(x); t2=doubleIt(y). Path constraints for the two branches at if(t1>10*z)?

✅ANSWER: Branch TRUE: (x0*x0>0 AND 2*y0>0) AND (x0*x0>10*z0) Branch FALSE: (x0*x0>0 AND 2*y0>0) AND (x0*x0<=10*z0)

→ t1 = squareIt(x) = x0*x0
→ t2 = doubleIt(y) = 2*y0
→ Condition 1: t1>0 && t2>0 --> x0*x0 > 0 AND 2*y0 > 0
→ At branching if(t1>10*z): TRUE --> x0*x0>10*z0, FALSE --> x0*x0<=10*z0

9.3 DART (Directed Automated Random Testing)

DART combines symbolic execution with random testing to find paths automatically.

→ 1. Start with random concrete values
→ 2. Run program and record path constraints
→ 3. NEGATE one constraint to force a new path
→ 4. Solve new constraints to get new test values

CHAPTER 10: Logic Coverage – RACC / RICC / CACC / GICC

 APPEARED IN: ALL EXAMS | HARDEST CHAPTER – 12+ Questions!

10.1 Key Definitions

Term	Meaning
Predicate (p)	A Boolean expression (TRUE or FALSE). e.g. p = (a AND b) OR NOT c
Clause (literal)	An atomic boolean expression within predicate. e.g. a, b, c are clauses
pa	The condition under which clause "a" DETERMINES the value of predicate p
RACC pair	Two rows that differ ONLY in clause "a" value, and a determines p in both
RICC pair	Two rows where a is inactive but p changes (Restricted Inactive Clause Coverage)
CACC	Correlated Active Clause Coverage - similar to RACC with correlated conditions

10.2 Understanding pa (When does clause a DETERMINE p?)

Clause a DETERMINES predicate p when CHANGING a's value (T->F or F->T) changes p's value.

Example: p = (a AND b) OR NOT c

pa = condition where "a" determines p = when b=T AND c=T (so NOT c = F, and rest does not mask a)

Q1. p = (a AND b) OR NOT c. Truth table given. Which are RACC pairs for clause b?

✅ANSWER: {(1,3)} - rows where b changes and b determines p

→ Find pb (when does b determine p?): For b to determine p, a must be T and NOT c must be F (c=T)

→ So pb = (a=T AND c=T)

→ Look at truth table rows where a=T, c=T: Row 1 (a=T,b=T,c=T,p=T) and Row 3 (a=T,b=F,c=T,p=F)

→ These rows differ ONLY in b, and pb holds --> RACC pair = {(1,3)}

 MEMORY TIP: RACC = rows that ONLY differ in the clause being tested, and that clause determines p

Q2. p = (a AND b) OR NOT c. RICC pairs for literal b?

✅ANSWER: {(2,4)} for p=T rows and {(5,7)} for p=F rows

→ RICC = Restricted INACTIVE Clause Coverage

→ Inactive means b does NOT determine p (pb is FALSE = when a=F or c=F)

→ Among those rows, find pairs where same values of other clauses, b changes value

 MEMORY TIP: RICC = rows where the clause is INACTIVE (does not determine p), but exists in pairs

Q3. p = a OR (b OR c). When does a determine p?

✅ANSWER: pa = when (b OR c) is FALSE = when b=F and c=F

→ p = a OR (b OR c). For a to determine p, the REST (b OR c) must = FALSE

→ b OR c = FALSE only when b=F AND c=F

 MEMORY TIP: To find "when does X determine p": set everything else to make predicate depend on X alone

Q4. p = a AND (NOT b OR NOT c). What represents pa?

✅ANSWER: pa = NOT b OR NOT c (condition where a determines p)

→ p = a AND (NOT b OR NOT c). For a to determine p, the rest must be TRUE

→ (NOT b OR NOT c) must be TRUE = at least one of b or c is FALSE

→ So pa = NOT b OR NOT c

CHAPTER 11: Component Expressions for Java Servlets

 APPEARED IN: ALL EXAMS | MEDIUM FREQUENCY – Learn the notation!

11.1 Component Expression Operators

Operator	Symbol	Meaning	Example
Sequence	(dot)	P1 executes, THEN P2	P1 . P2
Alternation (branch)	| (pipe)	Either P2 OR P3 executes (if-else)	P2 | P3
Iteration (loop)	P* (star)	P executes ZERO OR MORE times (loop body)	P3*

11.2 Reading a Servlet and Writing Component Expression

Typical pattern: P1 . (P2 | (P3* . P4 . (P5 | P6))) . P7

→ Sequential: P1 then P2 = P1 . P2
→ if(condition) { P2 } else { P3 } = P2 | P3
→ for(loop) { P3 } = P3*

Q1. Servlet: init P1 --> if(null) {P2} else {for-loop{P3} --> P4 --> if(condition){P5} else{P6}} --> P7. Component expression?

✅**ANSWER: P1 . (P2 | (P3* . P4 . (P5 | P6))) . P7**

→ Step 1: P1 executes first --> P1 .
→ Step 2: if(data==null) --> branch: P2 if true, or else-block if false
→ Step 3: else block = for-loop(P3*) . P4 . if-else(P5|P6)
→ Step 4: P2 OR (P3* . P4 . (P5|P6)) = P2 | (P3* . P4 . (P5|P6))
→ Step 5: Then P7 executes --> . P7

 MEMORY TIP: PATTERN: init . (null-case | (loop . compute . (branch1 | branch2))) . close*

Q2. In atomic section P3 (out.println("Score: " + total_marks + "<BR/>")), which is a content variable?

✅**ANSWER: total_marks**

→ total_marks is the COMPUTED VALUE being output to the user
→ "Score: " and "</BR>" are HTML strings, not content variables

 MEMORY TIP: CONTENT VARIABLE = the actual DATA variable being displayed. Not strings, not methods.

CHAPTER 12: Context-Free Grammars & Finite State Machines

APPEARED IN: 2023-Dec, 2024-Sep, 2024-Dec, 2024-Aug | MEDIUM FREQUENCY

12.1 Context-Free Grammars (CFG) – Step by Step

A CFG defines a language via PRODUCTION RULES. Start with S, apply rules until only terminals remain.

TERMINALS = actual characters (a, b, c). NON-TERMINALS = variables (S, X, A). Starting symbol = S

Q1. G: S-->bX, X-->bXa, X-->c. Over {a,b,c}. Language?

✅ANSWER: {b^(n+1) c a^n | n >= 0}

→ Start with S --> bX

→ Apply X-->bXa n times: S --> b^(n+1) X a^n

→ Apply X-->c: S --> b^(n+1) c a^n

→ n=0: S-->bX-->bc | n=1: S-->bbXa-->bbca

MEMORY TIP: GRAMMAR TRICK: Count how many times you apply the recursive rule = n.

Q2. G: S-->aXa, X-->bXb, X-->c. Language?

✅ANSWER: {a b^n c b^n a | n >= 0}

→ S --> aXa --> a(bXb)a --> a(b(bXb)b)a --> ... --> a b^n c b^n a

Q3. G: S-->aSa, S-->BB, B-->b. Language?

✅ANSWER: {a^n b b a^n} = {a^n bb a^n | n >= 0}

→ S-->BB-->bb (n=0) gives "bb"

→ S-->aSa-->a(BB)a-->abba (n=1)

→ S-->aSa-->a(aSa)a-->aa(BB)aa-->aabbaa (n=2)

Q4. G: S-->aSbb, S-->abb. Language?

✅ANSWER: {a^n b^(2n) | n >= 1}

→ Base: S-->abb = a b^2 (n=1)

→ S-->aSbb --> a(aSbb)bb --> aabbbb --> a^2 b^4 (n=2)

12.2 Finite State Machines (FSM)

FSM has: STATES (circles), TRANSITIONS (arrows with labels), START STATE, ACCEPT STATE (double circle)

A string is ACCEPTED if starting from start state, following transitions, ends in ACCEPT state

Q5. FSM with S1,S2 (accept),S3,S4. Transitions on a,b,d,e,c. Which strings accepted: abed/adec/abde/ad?

✅ANSWER: adec and ad (accepted) - trace through the given transitions

→ TRACE each string character by character through the FSM arrows

12.3 Regular Expressions

Pattern	Meaning	Example Strings
0 . (0+1)* . 1	Starts with 0, any 0s/1s, ends with 1	01, 001, 0101

(0+1)*	Any string of 0s and 1s	empty, 0, 1, 01, 10

Q6. Regular expression for all strings of 0s and 1s beginning with 0 and ending with 1?

✅**ANSWER: 0 . (0+1)* . 1**

→ Must start with 0: begin with literal 0

→ Any 0s and 1s in between: (0+1)* = zero or more repetitions of (0 or 1)

→ Must end with 1: end with literal 1

MEMORY TIP: (0+1) = any combination of 0s and 1s (including empty). Think of it as "wildcard for binary"*

CHAPTER 13: TDD & Software Quality Metrics

 APPEARED IN: 2024-Dec, 2025-Apr, 2025-Aug | LOW-MEDIUM FREQUENCY

13.1 Test-Driven Development (TDD)

TDD = Write TEST first, then write CODE to pass that test, then REFACTOR.

Cycle: RED (test fails) --> GREEN (make it pass) --> REFACTOR (clean up)

Q1. TDD for TriangleType::type. Which INVALID conditions must the implementation handle?

✅ANSWER: any side <= 0, OR triangle inequality fails (a+b <= c or similar)

→ INVALID if: (a<=0 || b<=0 || c<=0) OR (a+b<=c || a+c<=b || b+c<=a)

→ EQUILATERAL: a==b && b==c

→ ISOSCELES: a==b || a==c || b==c

→ else: SCALENE

 MEMORY TIP: Key: (1,2,3) fails triangle inequality: 1+2=3 which is NOT STRICTLY greater than 3

Q2. Why is TDD ideal for pair programming?

✅ANSWER: Because it allows one developer to write the failing test, and the other to write implementation to pass it

 MEMORY TIP: TDD + PAIR: One person writes the failing test, other makes it pass. Natural division!

13.2 Software Quality Metrics

Metric	Category	Memory Key
Defect Density	Product Quality / Software Metrics	Defects per unit of code
Cyclomatic Complexity	Product Quality / Software Metrics	Code complexity measure
Phase-Based Defect Removal Pattern	In-Process Quality Metrics	During development phases
Fix Backlog & Backlog Management Index (BMI)	Maintenance Quality Metrics	Unresolved issues backlog
Defect Removal Effectiveness (DRE)	In-Process Quality Metrics	% defects removed before release
Mean Time to Failure (MTTF)	Product Quality Metrics	How long before failure
Customer Satisfaction	Product Quality Metrics	User happiness score

Q3. Which metrics are product quality metrics?

✅ANSWER: Mean time to failure, Phase-based defect removal pattern, Satisfaction of customers

→ Product quality = metrics about the PRODUCT itself (defects, failures, user satisfaction)

→ Fix backlog = maintenance metric (managing unresolved issues)

Q4. Backlog Management Index (BMI) belongs to which category?

✅ANSWER: Maintenance Quality Metrics

 MEMORY TIP: BACKLOG = old unresolved issues. Managing them = Maintenance. BMI = Maintenance Quality Metric.

13.3 Module Coupling Types

Coupling Type	Description	Example
Parameter Coupling	Data passed as parameters	function(x, y)
Shared Data Coupling	Modules share global/common data	Global variables
External Device Coupling	Both modules access same external resource	Same file/device
Message-Passing Interfaces	Modules communicate via messages/queues	Message queues, events

Q5. Ride-sharing: User Module sends request to Driver Module via central MESSAGE QUEUES. Coupling type?

✅**ANSWER: Message-Passing Interfaces**

CHAPTER 14: Deployment Types & DNF Predicates

APPEARED IN: 2024-Sep, 2024-Dec | LOW FREQUENCY

Deployment Type	Description	Memory Key
Embedded	Installed on hardware device, usually no direct user communication	Hardware embedded = no screen
Shrink-wrap	Pre-installed on computer (bought at store)	Shrink-wrapped box at store
Contract	Purchaser pays developer to develop and install, fixed price	Custom contract work
Bundled	Installed by end-users themselves (downloaded/bought)	User installs it

14.2 Disjunctive Normal Form (DNF)

DNF = sum of products = ORs of ANDs. Form: (a AND b) OR (c AND d) OR e

NOT DNF: if there is an AND of ORs at the top level = that is CNF (Conjunctive Normal Form)

Q1. Which predicates are in DNF?

✅ **ANSWER: (a AND b) OR b OR c and (a AND b) OR (c AND d) OR e**

→ DNF = OR at the top level, AND inside. (a AND b) OR b OR c = yes DNF

→ (a OR b) AND (c OR d) = CNF, NOT DNF (AND of ORs at top level)

MEMORY TIP: DNF = think "sum of products" in digital circuits. OR of AND-terms.

□ CHAPTER 15: MEGA CHEAT SHEET & MIND MAP

THE ULTIMATE QUICK REFERENCE – STUDY THIS BEFORE EXAM!

⚡ QUICK FORMULAS

V(G) = E - N + 2 (Cyclomatic Complexity)

ECC Tests = Max block count across all characteristics

BCC Tests = 1 + sum of (blocks_i - 1) for each characteristic i

Mutation Score = Killed Mutants / Total Mutants x 100%

Coverage Strength: NC <= EC <= EPC <= PPC (weakest to strongest)

Data Flow: All-Defs <= All-Uses <= All-DU-Paths

□ MOST TESTED TOPICS (by frequency)

Rank	Topic	Appeared	Must Know
□ 1	Types of Testing (Performance/Spike/Soak)	6/6 exams	ALL types + definitions
□ 1	ATG Transitions (Simple/Form/Redirect/Op)	6/6 exams	All 4 types + examples
□ 1	OO Anomalies (SVA/SDA)	6/6 exams	SVA=hiding, SDA=no super call
□ 1	OO Mutation (PCD/SKD/OMD/HVI)	6/6 exams	All 4 OO operators
□ 1	Component Expressions (Servlets)	5/6 exams	P1.(P2|(P3*.P4.(P5|P6))).P7
□ 2	Cyclomatic Complexity	5/6 exams	V(G) = E - N + 2
□ 2	Logic Coverage (RACC/RICC)	5/6 exams	When clause determines p
□ 2	ISP / ECC / BCC	5/6 exams	ECC=max, BCC=1+extra
□ 2	Data Flow (DU-paths)	5/6 exams	def->use with no re-def
□ 2	Symbolic Execution	5/6 exams	x->x0, build constraints
□ 3	CFG Prime Paths	4/6 exams	Maximal simple paths
□ 3	Method Invocation in OO	4/6 exams	Dynamic dispatch from bottom
□ 3	Context-Free Grammars	4/6 exams	Trace production rules

□ ATG TRANSITION DECISION TREE

Question to Ask	Answer
Was it an AUTOMATIC redirect by server?	REDIRECT Transition
Did user CLICK A FORM SUBMIT button?	FORM LINK Transition
Did user click a hyperlink or edit URL?	SIMPLE LINK Transition
Was it browser Back/Forward button?	OPERATIONAL Transition

□ ANOMALY DECISION TREE

Symptom	Cause	Anomaly Type

Variable has SAME NAME in parent and child, child value does not reach parent print()	Child re-declares variable (HIDING)	SVA - State Visibility Anomaly
Override method does not call super, so parent variable is never set (stays 0/default)	Missing super.method() call	SDA - State Definition Anomaly
Wrong TYPE used for a variable	Type mismatch	ITU - Inconsistent Type Use

▯ MUTATION OPERATOR QUICK REFERENCE

Change Seen	Operator
Math operator changed (+->-, *->/, %->/)	Arithmetic Operator Replacement (AOR)
Comparison changed (>->=, ==->!=)	Relational Operator Replacement (ROR)
Variable name changed (a->b, x->y)	Scalar Variable Replacement (SVR)
super() in constructor deleted	Parent Constructor Deletion (PCD)
super.method() deleted (non-constructor)	Super Keyword Deletion (SKD)
Overriding method body deleted	Overriding Method Deletion (OMD)
Argument positions swapped (a,b)->(b,a)	Argument Order Change (AOC)
Overloaded method body changed	Overloading Method Change (OMC)

▯ COVERAGE CRITERIA QUICK REFERENCE

Criterion	Strength	Key Requirement
Node Coverage (NC)	Weakest	Visit every node at least once
Edge Coverage (EC)	Weak	Traverse every edge at least once
Edge-Pair Coverage (EPC)	Medium	Every consecutive edge pair (3 nodes)
Prime Path Coverage (PPC)	Strong	Every prime path (with side trips OK)
All-Defs Data Flow	Weak	Every definition reaches some use
All-Uses Data Flow	Medium	Every def-use PAIR covered
All-DU-Paths Data Flow	Strong	Every distinct DU-PATH covered
RACC Logic	Strong	Active clause determines predicate
RICC Logic	Strongest	Inactive clause coverage restricted

▯ MIND MAP – Software Testing

SOFTWARE TESTING
- FUNCTIONAL: Unit | Integration | System | Acceptance | Beta | Regression
- NON-FUNCTIONAL: Performance | Scalability | Load | Stress | Spike | Soak | Security | Compatibility
- WHITE BOX:
  - Control Flow --> CFG, Node/Edge/Prime Path coverage
  - Data Flow --> Def-Use pairs, DU-Paths (All-Defs, All-Uses, All-DU-Paths)
  - Logic Coverage --> RACC, RICC, CACC, GICC
  - Mutation Testing --> Basic (AOR,ROR,SVR,SDL) + OO (PCD,SKD,OMD,HVI)
  - Symbolic Execution --> DART technique
- BLACK BOX:
  - Input Space Partitioning --> ECC (max), BCC (1+extra)

Boundary Value Analysis --> BVA, Robust BVA, Worst Case BVA
TDD --> Red -> Green -> Refactor

WEB TESTING:
ATG --> Simple Link | Form Link | Redirect | Operational
Client-side --> Value | Parameter | Control Flow | Session
Component Expressions --> P1.(P2|(P3*.P4.(P5|P6))).P7
Web Layers --> Presentation | Content | Representation | Storage

□ LAST-MINUTE MEMORY MNEMONICS

Topic	Mnemonic
ATG Transitions: Simple, Form, Redirect, Operational	"Some FRiends are Outstanding" - S F R O
OO Anomalies: SVA, SDA, SDIA, ITU	"Some Students Do It Too" - SVA SDA SDIA ITU
OO Mutations: PCD, SKD, OMD, HVI	"Please Stop Opening Hatches" - PCD SKD OMD HVI
Test Types: Load, Stress, Spike, Soak	"Low Stress Spikes Soak" - normal, beyond, sudden, long
Bypass Testing: Value, Parameter, Control, Session	"Very Possible Control Session" - VPCS
Coverage: NC < EC < EPC < PPC	"Never Eat Enough Pizza Please" - N E E P P

□ FINAL EXAM STRATEGY

→ 1. READ ALL OPTIONS before selecting. Many questions have tricky options.
→ 2. ATG questions: Ask "Was it automatic?" --> Redirect. "Form submit?" --> Form Link.
→ 3. Anomaly questions: Look for hidden variables (SVA) vs missing super call (SDA).
→ 4. ISP questions: ECC = max block count. BCC = 1 + (extra blocks per characteristic).
→ 5. Cyclomatic: V(G) = E - N + 2. Count arrows and circles carefully.
→ 6. Symbolic execution: Replace variable with symbol, trace path, AND all constraints.
→ 7. Logic coverage: Find pa first (when does clause determine predicate?), then find rows.
→ 8. OO mutations: super() in constructor = PCD. super.method() = SKD. Variable hidden = HVI.
→ 9. Component expressions: if-->|, loop-->*, sequence-->. Build from inside out.
→ 10. Grammar questions: Trace 2-3 derivations to find the pattern.

YOU'VE GOT THIS! □

Study this guide. Practice the PYQs. Score 100!

SUMIT GUPTA MADE THIS PDF WITH USE OF AI
All the best for your Software Testing exam! □